Recurrent networks

Chicago transit ridership

LECTURE 16 | GÉ\;RON CH. 13

Applications of Machine Learning

BSc Mathematics of Artificial Intelligence ·\; Third year

Where we are

Lecture 15 established the ground rules and a number to beat:

- Evaluation is a **rolling origin**, never a random split
- The seasonal-naive forecast — copy the same weekday last week — is genuinely hard to beat, and the autocorrelation says why
- A linear model on lagged features is an AR model, and it does beat the baseline, modestly

> **WHAT A LINEAR MODEL ON LAGS CANNOT DO**
>
> It has a **fixed window**. Anything outside the last 56 days is invisible to it, and every lag gets one weight regardless of context. A recurrent network keeps a state instead, and that state is not bounded by a window.

Today

1. Recurrent cells, unrolling, and backpropagation through time (20 min)
2. **The method** — deep RNNs, multivariate inputs, and forecasting several steps ahead (35 min)
3. LSTM and GRU, gate by gate (20 min)
4. A convolutional alternative, and what we still cannot do (15 min)

No new derivation today. Lecture 11's variance argument is the one that applies, unrolled through time instead of down layers — which is exactly why the gates exist. the architectures, and what each gate is for.

EXAMINABLE

FIRST TWENTY MINUTES

A cell with a memory

20 minutes ·\; examinable

The recurrent cell, written out

$$\mathbf{h}_t = \phi\left(\mathbf{W}_x^\mathsf{T}\mathbf{x}_t + \mathbf{W}_h^\mathsf{T}\mathbf{h}_{t-1} + \mathbf{b}\right)$$

ONE STEP OF A RECURRENT LAYER

- $\mathbf{x}_t$ — the input at step t; $\mathbf{h}_t$ — the state after it
- $\mathbf{W}_x$ and $\mathbf{W}_h$ do not depend on t: **the same weights at every step**
- Compare Lecture 9: this is $\phi(\mathbf{X}\mathbf{W} + \mathbf{b})$ with one extra term, and that term is the whole idea

The state is the only route by which anything before step t reaches step t. Its width is therefore a budget: whatever the network needs to remember has to fit in $\mathbf{h}$.

Why not just a longer window

A linear model on 56 lags has 56 weights. Take 365 lags and it has 365 — each fitted from the same amount of data, each learning one number about one position.

- The parameter count grows linearly with the reach
- Nothing is shared between positions: the weight on lag 7 learns nothing from the weight on lag 14, though both are “a week ago”
- Anything beyond the window is invisible, however important

> **WHAT RECURRENCE CHANGES**
>
> One set of weights, applied at every step. Reach is decoupled from parameter count — the same argument as weight sharing in Lecture 12, along time instead of across an image.

The state is a summary

$\mathbf{h}_t$ is everything the network has decided to remember about $x_1 \ldots x_t$.

- It is **fixed width**, whatever the sequence length — so it is a lossy summary by construction
- What it keeps is learned, not designed: nothing tells the cell that “the same weekday last week” matters
- Two sequences with the same state are indistinguishable to everything downstream. That is the definition of a sufficient statistic, and the network is trying to learn one

Which is also the honest limit: if the task needs more bits than the state has, no amount of training supplies them.

Why not simply stack dense layers?

You could. Flatten the 56 values and put a multilayer perceptron on top. It would work.

- But the window length is baked into the first layer's shape
- Nothing in the architecture knows that position 40 comes before position 41
- Every position gets its own private set of weights, so a pattern learned at one position is not available at another

The same argument as convolution against dense layers on images, transposed from space to time.

A recurrent neuron

A neuron that, at every step, receives the input *and* its own output from the previous step.

$$\mathbf{h}_t = \phi(\mathbf{W}_x^\intercal \mathbf{x}_t + \mathbf{W}_h^\intercal \mathbf{h}_{t-1} + \mathbf{b})$$

- $\mathbf{h}_t$ — the hidden state after step t
- $\mathbf{W}_x, \mathbf{W}_h$ — two weight matrices, shared across **every** time step
- ϕ — the activation\; $\tanh$ by default, and there is a reason

The hidden state is a function of every input so far. That is the memory.

Unrolling through time

Draw the same cell once per time step and you get a network 56 layers deep — but with **one** set of weights, reused.

> **TWO CONSEQUENCES, BOTH IMPORTANT**
>
> The parameter count does not grow with the sequence length. And the gradient has to travel back through 56 applications of the same matrix, which is the variance-propagation problem from Lecture 14 in a new costume.

That second consequence is why $\tanh$ rather than ReLU: a bounded activation keeps the repeated product from exploding.

Four shapes of recurrent model

Shape	Input	Output	Example
sequence to vector	a sequence	one value	**✓ today's model**
vector to sequence	one value	a sequence	a caption from an image
sequence to sequence	a sequence	a sequence	a forecast at every step
encoder–decoder	a sequence	a sequence	translation

We build the first today. The third appears in the next lecture, and the fourth in two applications' time.

Training it: backpropagation through time

Unroll the network over the window and it becomes a deep feed-forward network whose layers *share their weights*. Then differentiate it exactly as Lecture 10 taught — nothing new is needed.

> **WHICH IS ALSO THE PROBLEM**
>
> Unrolling 56 steps gives a 56-layer network, and Lecture 11 said what happens to a gradient descending 56 layers: it is multiplied by the same factor 56 times. Here the factor is the *same* matrix every step, because the weights are shared — so the gradient is governed by a power of one matrix, and powers of a matrix either vanish or explode unless its spectral radius sits almost exactly at one.

Initialisation cannot fix this the way it did in Lecture 11, because there the layers were independent. Something has to change in the cell itself.

The model, in eleven lines

```
import torch.nn as nn

class SimpleRnn(nn.Module):
    def __init__(self, input_size=1, hidden_size=32, output_size=1):
        super().__init__()
        self.rnn = nn.RNN(input_size, hidden_size, batch_first=True)
        self.head = nn.Linear(hidden_size, output_size)

    def forward(self, X):                       # X: [batch, time, series]
        outputs, last_state = self.rnn(X)
        return self.head(outputs[:, -1])        # only the final step
```

`batch_first=True` because our batches are [batch, time, series]. Without it the module silently reads the first dimension as time.

Why the linear head is not optional

- The hidden state has 32 components\; the target has one
- $\tanh$ outputs lie in $(-1, 1)$, and our scaled targets sometimes exceed that
- The head is one affine map from 32 numbers to one, applied to the last step only

> `outputs[:, -1]` is the hidden state after the final time step. Everything the model knows about the window has to fit in those 32 numbers.

Training it

```
model = SimpleRnn()
loss_fn = nn.HuberLoss(delta=0.05)
opt = torch.optim.SGD(model.parameters(), lr=0.05, momentum=0.9)
loader = DataLoader(train_set, batch_size=32, shuffle=True)

for epoch in range(200):
    model.train()
    for Xb, yb in loader:
        opt.zero_grad()
        loss_fn(model(Xb), yb).backward()
        opt.step()
```

The same five lines as Lecture 12, unchanged. `zero_grad` is still not optional and still fails silently.

Why the Huber loss

$$L_\delta(e) = \begin{cases} \frac{1}{2}e^2 & |e| \le \delta \\ \delta\left(|e| - \frac{1}{2}\delta\right) & |e| > \delta \end{cases}$$

Quadratic near zero, linear far away. We are optimising a proxy for MAE, and minimising the absolute error directly gives a gradient that never gets smaller as you approach the optimum.

The metric and the training loss are allowed to differ. Lecture 2 said so\; here is the case.

Shuffle, but shuffle the right thing

> `DataLoader(train_set, shuffle=True)` shuffles the **windows**. It does not shuffle the values inside a window — the sequence within each row is untouched.

Gradient descent expects the instances in a batch to be independent draws. Presenting 56 consecutive days as a batch, in order, gives 56 highly correlated gradients and a noisier step, not a smoother one.

Read that sentence again in the next lecture.

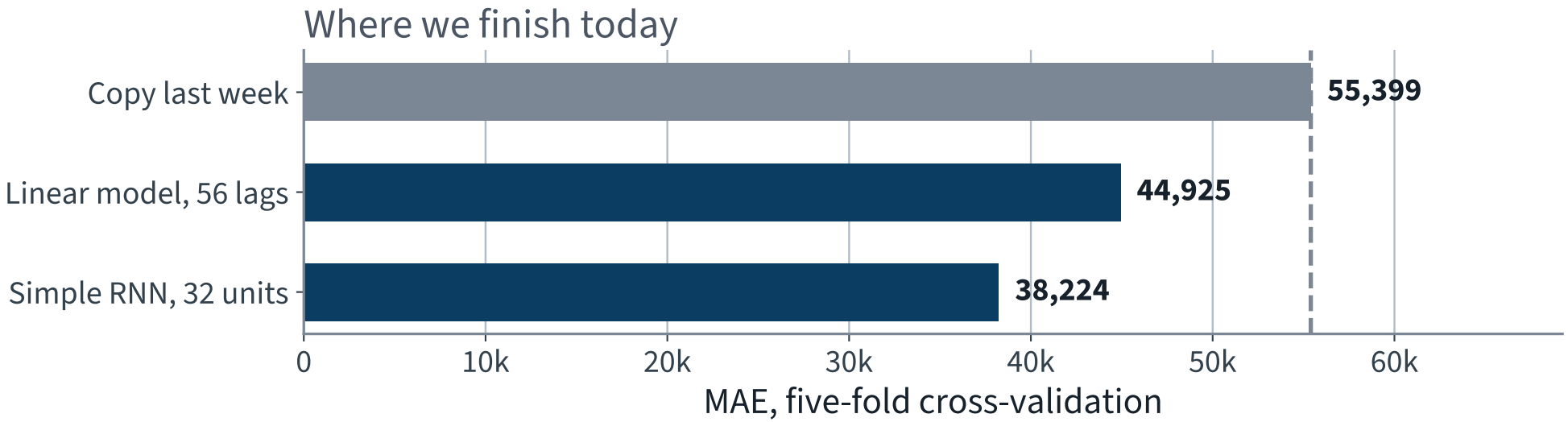
The recurrent network, measured the same way

38,224

Five folds, shuffled, seed 42 — identical protocol to the linear model.

Against **55,385** for copying last week on the same folds and **44,925** for the linear model. That is **31.0%** better than the baseline, and 15% better than the linear model on the same folds.

Where we finish today



Two models, both comfortably past the baseline, on identical folds.

Reading it

- Both models beat the baseline by more than the fold-to-fold spread, so the ordering is real
- The recurrent model beats the linear one — it can use the *order* of the window, not only a weighted sum of it
- Neither is close to zero, and neither should be: some of the next day is not in the last eight weeks

Everything on this slide is a statement about the folds we measured on. Every such statement carries an unstated assumption, and ours is on the next slide but one.

THE METHOD

Deeper, wider, and further ahead

35 minutes ·\; examinable

The notebook for this lecture

Open Lecture 16 in Colab

- **Runs on free CPU.** The series is short and the networks are small\; every model here trains in seconds
- Every architecture scored on the *same* rolling-origin protocol as Lecture 15, so the ladder is a comparison and not a collection
- The gradient norm through an unrolled RNN, measured — the same probe as Lecture 11, along time instead of depth

> **WHAT TO CHANGE**
>
> Lengthen the window from 56 steps to 200 and re-run the plain RNN. It gets worse, not better, and the gradient probe says why.

Four shapes of sequence problem

Shape	In	Out	Example
sequence to vector	a sequence	one value	forecast one step; classify a review
vector to sequence	one value	a sequence	caption an image
sequence to sequence	a sequence	a sequence	forecast seven days; translate
encoder–decoder	a sequence	a sequence, after reading all of it	translate properly

The cell is the same in all four. What changes is where you take the output from — the last state, every state, or a second network fed by the last state — and that choice is the architecture.

Reading the shapes off the code

- `return_sequences=False` — one output, from the final state. Sequence to vector
- `return_sequences=True` — one output per step. Needed for every layer that feeds another recurrent layer, or you hand the next layer a vector where it expects a sequence
- A `Dense` on top applied at every step is *time-distributed*: the same weights at every position, which is the recurrent idea again, one level up

> **THE COMMONEST SHAPE ERROR**
>
> Stacking two recurrent layers and forgetting `return_sequences=True` on the first. It does not raise: the second layer receives a *length-one* sequence and trains happily on almost no information. The symptom is a model that is inexplicably no better than a shallower one.

Loss for a forecast

Squared error punishes a large miss quadratically, which on a series with occasional shocks means a handful of days dominate the gradient.

Loss	Behaviour
MSE	smooth everywhere; dominated by outliers
MAE	robust; gradient is constant, so it converges slowly near the optimum
Huber	quadratic near zero, linear beyond — both properties, one knob

The choice is the same argument as RMSE against MAE in Lecture 2, and it has the same answer: it depends on what an error costs the operator, and that is a question for them and not for us.

Windows overlap, and that matters

Consecutive training windows share almost all their content: a 56-day window starting on day t and one starting on day $t + 1$ differ in two values out of fifty-six.

TWO CONSEQUENCES

Shuffle them. Otherwise a mini-batch is fifty near-identical examples, its gradient is one example's gradient with less noise, and training stalls.

Do not let that shuffling cross the split. Windows are shuffled *within* the training period only; the evaluation period is still strictly later in time. Shuffling and leakage are different operations and it is easy to conflate them.

Normalising a series

- Scale by a constant computed on the **training period only** — the same rule as every other lecture, and easier to break here because the series looks like one object
- Divide rather than standardise if the series is positive and multiplicative: ridership is counts, and a ratio is more natural than a difference
- If you differenced, remember to *un*-difference the prediction before reporting it. The metric is in the units the operator uses

✗ A model evaluated in differenced units can look excellent and mean nothing: predicting a change of zero every day is nearly right, and it is the naive forecast wearing a disguise.

What to log while it trains

- Training and validation loss per epoch — the gap is Lecture 5's
- The gradient norm — Lecture 11's probe, along time here rather than depth. A plain RNN on a long window shows it decaying
- The forecast itself, plotted against truth, on a held-out stretch. A number cannot tell you the model has learned to output last week's value with a one-day lag; a picture can

> **THE FAILURE A PLOT CATCHES AND A METRIC DOES NOT**
>
> A model that simply repeats the previous value scores well on almost any error metric and is worthless. On a plot it is unmistakable: the prediction is the truth, shifted right by one step.

Forecasting k steps ahead: three strategies

Strategy	How	Weakness
Recursive	predict one step, feed it back, repeat	errors compound\; the model never trained on its own output
Direct	a separate model per horizon	k models, and none of them shares what the others learned
Sequence-to-sequence	one model, k outputs at once	more to fit, and the outputs are not independent

All three are defensible and all three must be evaluated the same way: **on the rolling origin, at the horizon you actually need**. A model that is excellent one step ahead and useless seven steps ahead is not a good model if the operator plans a week in advance.

Teacher forcing, and what it costs

Training a sequence-to-sequence model, you can feed the decoder the *true* previous token or its *own* previous prediction.

	Trains	At prediction time
True token (teacher forcing)	**fast and stable ✓**	**✗ the model has never seen its own mistakes**
Own prediction	**✗ slow, and early training is noise**	**matches deployment ✓**

EXPOSURE BIAS

A model trained only on true prefixes meets a distribution at prediction time that it never saw in training: its own errors, compounding. The forecast drifts, and the further ahead you go the worse it gets — which is precisely the recursive strategy's weakness two slides ago.

Bidirectional layers

Run one recurrent layer forwards and another backwards, and concatenate their states. Every position then sees the whole sequence.

> **THE CONDITION, AND IT IS ABSOLUTE**
>
> A backward pass reads the *future*. That is fine when the whole sequence is available at prediction time — classifying a review, tagging a sentence — and it is **leakage** when it is not. A bidirectional forecaster is Lecture 15's random split wearing a different hat: the number goes up and the system cannot be deployed.

Ask one question: *at prediction time, do I have the rest of the sequence?* For text classification, usually yes. For forecasting, never.

Carrying the state between batches

- **Stateless** — the state resets to zero for every window. Windows can be shuffled, which is what makes training well behaved
- **Stateful** — the final state of one batch becomes the initial state of the next. In principle unbounded memory\; in practice the batches must be consecutive and correctly ordered, so you cannot shuffle

> **THE TRADE, PLAINLY**
>
> Stateful buys memory beyond the window and costs you shuffling — and shuffling is what stops consecutive, highly-correlated windows from arriving in the same batch. Most practical systems are stateless with a long enough window, which is a statement about engineering rather than about theory.

Improvement 1 ·\; A deeper network

```
self.rnn = nn.RNN(input_size, hidden_size, num_layers=3,
                  batch_first=True)
```

One argument. Three recurrent layers stacked, the output of each feeding the next.

Model	Forecast MAE
Simple RNN, one layer	**49,073 ✓**
Deep RNN, three layers	**✗ 53,491**

✗ It got worse.

Why the deeper network lost

- Three times the parameters, fitted on the same 1,040 windows
- Its error on the training rows is *higher* too — 40,630 against 35,972 — so this is not overfitting, it is a harder optimisation
- Gradients now traverse 56 time steps *and* three layers, which is the vanishing-signal problem of Lecture 14 compounded

✓ Report the negative result. A ladder of improvements in which every rung works is a ladder somebody edited.

Improvement 2 ·\; Give it more series

We have been forecasting rail from rail alone, while a bus series sits unused beside it — and a day-type column we established is known in advance.

```
mulvar = df[["rail", "bus"]] / 1e6
mulvar["next_day_type"] = df["day_type"].shift(-1)   # known in advance
mulvar = pd.get_dummies(mulvar, dtype=float)         # 5 columns

assert list(mulvar.columns) == ["rail", "bus", "next_day_type_A",
                                "next_day_type_U", "next_day_type_W"]
```

The architecture changes by one number: `input_size=5`.

Wait — that is a `shift(-1)`

The previous lecture spent ten minutes on a `shift(-7)` that read the future. This one is deliberate.

> **THE TEST THAT SEPARATES THEM**
>
> Is the value **available at prediction time**? The type of the day we are forecasting is on the public calendar and has been for a year. Its ridership is not.

✗ This is the one question about leakage that no static check can answer for you. It is not a property of the code\; it is a property of the world.

The result

Model	Forecast MAE	vs copying last week
Copy last week	64,815	—
Simple RNN, rail only	49,073	24% better
Simple RNN, five inputs	**40,046 ✓**	38% better

✓ Eighteen per cent off the error for one line of feature engineering and one changed constructor argument.

Most of it is the day type. The network no longer has to infer that a holiday is coming from the shape of the previous eight weeks — which is impossible — because we told it.

Improvement 4 ·\; Forecasting further than one day

The operations team also wants a fortnight. Two ways:

Feed the forecast back

Predict day $t+1$, append it to the window as though it had happened, predict $t+2$. Errors accumulate.

Predict all fourteen at once

Change the output layer to fourteen units and the target to a vector. One model, no accumulation.

```
target = self.series[end : end + 14, 0]     # 14 values, not 1
model = GruModel(input_size=5, hidden_size=32, output_size=14)
```

And a third way — predict fourteen at *every* step

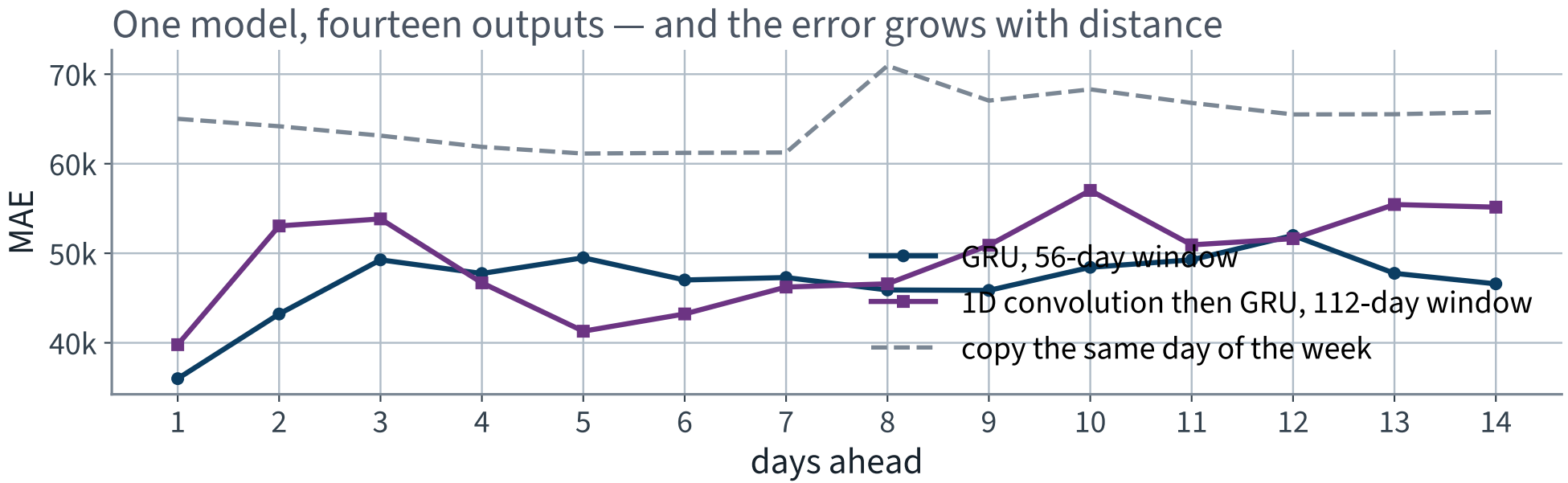
At step 0 the model outputs the forecasts for steps 1 to 14\; at step 1, for steps 2 to 15\; and so on. The target is a sequence of windows rather than a single vector.

```
def forward(self, X):
    outputs, _state = self.rnn(X)
    return self.head(outputs)        # every step, not outputs[:, -1]
```

> **IS THAT CHEATING?**
>
> No. A recurrent cell at step t has seen steps 1 to t and nothing more — it is **causal**. The loss simply gains a term at every step, so many more gradients flow and they travel a shorter distance.

After training, only the last step's output is used.

How far ahead can it see?



The error grows with distance, from **35,997** at one day to **46,586** at fourteen — and stays below the no-model baseline throughout.

Gradient clipping, here

Lecture 11 introduced clipping as one repair among seven. On a recurrent network it is not optional.

> **WHY HERE IN PARTICULAR**
>
> The gradient through T steps carries a power of one matrix. When its spectral radius exceeds one even slightly, the norm does not drift upwards — it explodes, over a handful of steps, and a single bad batch can move every weight to a place training never recovers from.

Clip by global norm, not per parameter: the direction of the update is information and only its length is the problem.

THE REPAIR

LSTM and GRU, gate by gate

20 minutes ·\; examinable

Why a gate helps, in one line

A plain cell *multiplies* the state at every step:

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}} = \operatorname{diag}(\phi')\,\mathbf{W}_h^{\mathsf{T}}$$

so over T steps the gradient carries T copies of the same matrix, and Lecture 11's argument applies with the extra force that it is *one* matrix rather than many independent ones.

WHAT A GATED CELL DOES INSTEAD

An LSTM keeps a separate cell state $\mathbf{c}_t$ updated **additively**: $\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t$. When the forget gate $\mathbf{f}_t$ is near one, $\partial \mathbf{c}_t / \partial \mathbf{c}_{t-1} \approx \mathbf{1}$ and the gradient passes through almost unchanged. That is the entire trick: **a path along which the gradient is added to rather than multiplied.**

LSTM, gate by gate

Gate	Equation	What it decides
forget $\mathbf{f}_t$	$\sigma(\cdot)$	how much of the old cell state to keep
input $\mathbf{i}_t$	$\sigma(\cdot)$	how much of the candidate to write
candidate $\tilde{\mathbf{c}}_t$	$\tanh(\cdot)$	what to write, if anything
output $\mathbf{o}_t$	$\sigma(\cdot)$	how much of the cell state to expose as $\mathbf{h}_t$

Every gate is a sigmoid of the same two inputs $(\mathbf{x}_t, \mathbf{h}_{t-1})$ with its own weights, so a sigmoid in $[0, 1]$ is a soft switch and the network *learns when to open it*. Four gates means roughly four times the parameters of a plain cell.

State what each gate does and why the cell-state update is additive.

EXAMINABLE

GRU: the same idea, two gates

- **Update gate** $\mathbf{z}_t$ — merges LSTM's forget and input into one: what is kept is what is not written
- **Reset gate** $\mathbf{r}_t$ — how much of the previous state to use when forming the candidate
- No separate cell state: $\mathbf{h}_t$ does both jobs

$$\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t$$

Still additive, still a near-identity path when $\mathbf{z}_t$ is small — the property that mattered, kept, with about three quarters of the parameters.

> **WHICH TO USE**
>
> On most problems they are within noise of one another. GRU is smaller and trains faster\; LSTM has the longer track record. Measure, on your own protocol, rather than inheriting an opinion.

Improvement 3 ·\; The problem with a plain recurrent cell

Unrolled over 56 steps, the network applies the same weight matrix 56 times. The gradient carries a product of 56 Jacobians.

> That is Lecture 14's variance-propagation argument with depth replaced by time. The signal from step 1 has been transformed 55 times before it reaches the output, and what survives is a memory that fades.

The repair is not a better initialisation. It is a cell that can **choose** what to keep.

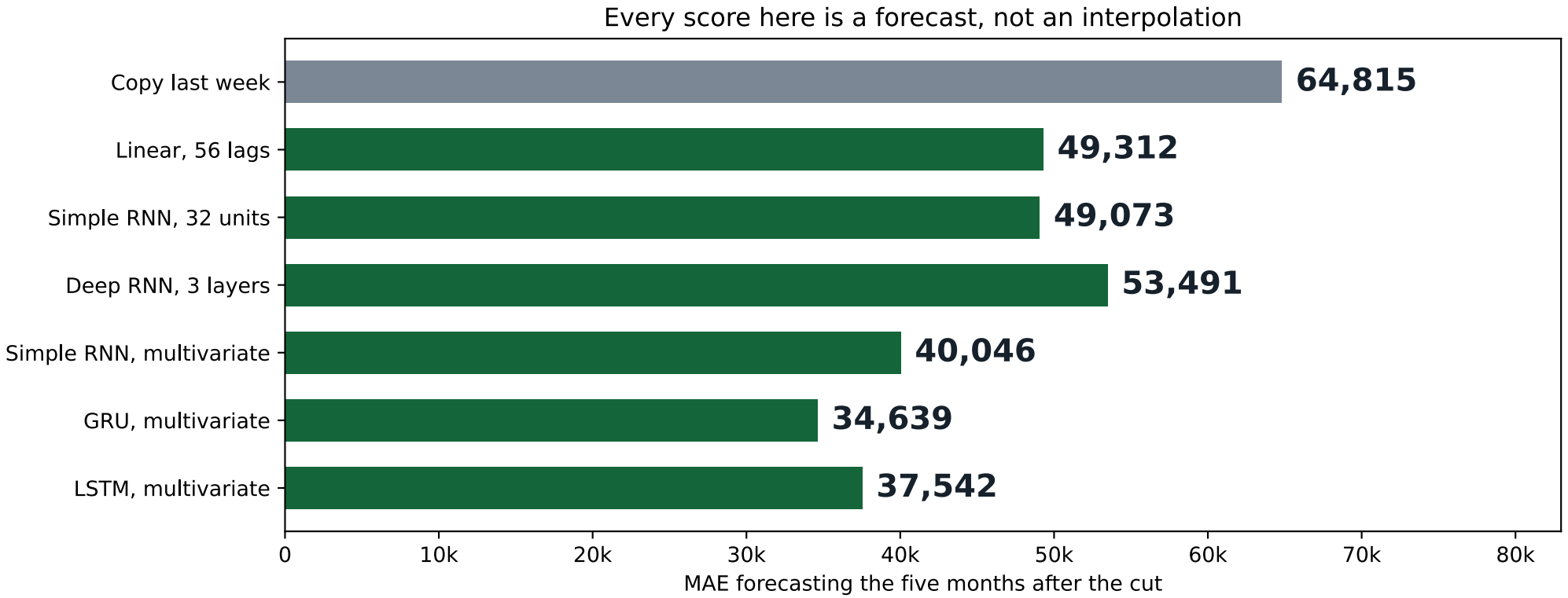
Both gated cells, measured

Model, all five inputs	Forecast MAE	Training MAE
Simple RNN	40,046	27,035
LSTM	37,542	24,540
GRU	**34,639 ✓**	**✗ 20,356**

The GRU wins on the forecast, and the second column says why to be careful: it fits the training rows 40% better than the plain cell but the forecast only 14% better. **That gap is overfitting**, and it is the next thing to attack.

Both cells are Chapter 13. Neither is a large model: a few thousand parameters.

Every model, measured the honest way



From 64,815 to 34,639 — **47%** better than doing nothing, and every bar is a forecast on the same days.

A convolutional alternative

A one-dimensional convolution with stride 2 halves the sequence before the recurrent layer sees it, so the same number of recurrent steps covers twice as much history.

```
self.conv = nn.Conv1d(input_size, hidden_size, kernel_size=4, stride=2)
self.gru  = nn.GRU(hidden_size, hidden_size, batch_first=True)
# time is a spatial dimension for Conv1d, so permute before and after
```

Model, 14 outputs	MAE at $t+1$	MAE at $t+14$
GRU, 56-day window	**35,997 ✓**	**46,586 ✓**
Conv then GRU, 112-day window	39,792	55,148

✗ On this series the extra history does not pay for the resolution it costs. Reported because we measured it, not because it won.

Dilated convolutions

A convolution with gaps: stack layers whose dilation doubles — 1, 2, 4, 8, 16 — and the receptive field grows exponentially with depth while the parameter count grows linearly.

	Recurrent	Dilated convolution
Along the sequence	**✗ sequential**	**parallel ✓**
Reach	in principle unbounded	fixed by depth and dilation
Gradient path	through every step	**through $\log T$ layers ✓**

It must be **causal**: pad on the left only, so no output depends on a later input. A symmetric convolution here is Lecture 15's leak again, in a third costume.

What we cut, and where it sits

Left out	Where it belongs
SARIMA and its grid search	Chapter 13 — in scope, cut for time
WaveNet: stacked dilated convolutions	Chapter 13 — in scope, cut for time
Layer normalisation inside a cell\; dropout between layers	Chapter 13
Attention over the window	Chapters 14–15, in two lectures
Prediction intervals rather than point forecasts	✗ **outside the book**

Thirty-five minutes buys four improvements measured end-to-end. It does not buy eight described end-to-end and none measured.

The whole application, in one table

Forecast	What we reported	What it does
Copy last week	55,385	64,815
Linear, 56 lags	44,925	**✗ 56,446**
Simple RNN	38,224	**✗ 49,073**
GRU, five inputs	—	**34,639 ✓**

The two middle rows are the same models measured twice. The bottom row is the model we would actually deploy, and it was never measured the wrong way.

✓ The final number is better than the one we committed to — but we reached it by first admitting that the committed one was fiction.

Is 34,639 good?

It is **47%** better than the system a transit authority could run with no data scientist at all, and its typical miss is around six per cent of a day's ridership.

> **WHAT A DEFENSIBLE REPORT SAYS**
>
> &ldquo\;Forecasting the five months after the training cut, and never seeing them, the model's mean absolute error is 34,639 boardings a day against 64,815 for copying the same day of the previous week. Both figures are on identical rows. The gain is concentrated on holidays.&rdquo\;

Note what that sentence contains: the protocol, the baseline, the rows, and where the gain comes from. Four things the number alone does not carry.

Red team it: what happens when the world changes?

Fit on everything to the end of 2019. Forecast the four months from March 2020, which nobody in this room needs reminding about.

	MAE
The linear model, trained to the end of 2019	✗ 84,380
Copying last week, on the same 122 days	36,282 ✓
The actual mean level over that period	141,404

✗ The model is 2.3 times worse than doing nothing, and its error is six tenths of the entire level.

And the shuffled split cannot see it at all

Run the same shuffled five-fold cross-validation over 2016–2021, spanning the collapse:

36,554

It reports a *better* number than before, because the post-2020 days are easier to interpolate: they are flat, low, and surrounded by other flat low days that are in the training set.

> **THE FAILURE MODE TO REMEMBER**
>
> A random split does not merely overstate performance. It is **structurally incapable** of detecting a regime change, which is the failure that actually ends forecasting systems.

To be fair to the model

March 2020 is not a leak and it is not the model's fault. No forecaster could have produced it from eight weeks of history, and copying last week wins only because it adapts within a week of the collapse.

> The lesson is not &ldquo\;the model is bad&rdquo\;. It is that an honest protocol *told you* the model had failed, on the day it failed, and the dishonest one reported an improvement.

✓ Which is why a deployed forecaster is monitored against its own naive baseline, continuously. If the margin disappears, something has changed in the world.

Temporal leakage checklist — keep this

- Is the split by **date**, not by row?
- Does every feature exist at the moment the forecast is made? Name the moment
- Were any statistics — means, scalers, encodings — computed over the whole series before the cut?
- Do the training windows overlap the scored windows? Purge them
- Is the baseline computed on *identical rows* to the model?
- Does the reported score come from more than one origin?

Six questions. The third is the leak from Lecture 2, meeting the leak from today.

What recurrence still cannot do

- **It is sequential.** Step t needs step $t-1$, so the computation cannot be parallelised along the sequence — on long sequences this, not the mathematics, is the binding constraint
- **The state is a bottleneck.** Everything the network remembers must fit in one fixed-width vector, whatever the sequence length
- **Long-range dependence is still hard.** Gates make the gradient survive\; they do not make position 400 as accessible as position 2

> **WHAT REPLACES IT**
>
> Let every position look directly at every other, with learned weights, and all three problems go at once: the computation parallelises, there is no single state to squeeze through, and position 400 is exactly as far away as position 2. That is attention, and it is Lecture 18.

What we did not do

- **Hierarchical or multi-scale models**, which forecast a weekly aggregate and a daily deviation separately
- **Probabilistic forecasts** — an interval rather than a point. Operators usually want one, and every method here gives a point
- **Exogenous shocks** known in advance: a public holiday, a closure, a fare change. The model can be told, and was not

All three are standard practice and none is examinable. They are here so you know they exist when a real forecasting problem asks for them.

BEYOND THE SYLLABUS, FOR CONTEXT

Reading a recurrent result honestly

Question	Why it decides whether the number means anything
At what horizon?	one step ahead is a different problem from seven
Against which baseline?	seasonal-naive, not the mean — Lecture 15
On which protocol?	rolling origin, and how many folds
With what spread?	one origin is one sample
In which units?	differenced or not; scaled or not

✗ A recurrent network that beats the naive forecast by less than the fold-to-fold spread has not beaten it. This is the paired comparison of Lecture 2, and it applies unchanged.

A recipe for a forecasting model

1. Plot the series. Seasonality and trend are usually visible, and cost nothing to find
2. Compute the autocorrelation, and the naive and seasonal-naive baselines. **These are the numbers to beat**
3. Fix the protocol before fitting anything: horizon, rolling origin, number of folds, metric
4. Fit a linear model on lags. It is fast, and it is often close
5. Only then a recurrent model, scored on the same protocol
6. Report the ladder with its spread, not the best row alone

✗ Steps 2 and 3 come before step 4 for the same reason they did in Lecture 2: after you have a number, you will not choose the protocol that makes it look worse.

What still cannot be parallelised

The sequential dependence is not an implementation detail that a better library removes.

$$\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{x}_t)$$

- Step t cannot begin until step $t-1$ has finished, by definition
- Batching parallelises across *sequences*, never along one
- So training time grows linearly with sequence length, on any hardware

On a 56-step window nobody cares. On a document of ten thousand tokens it is the binding constraint, and it is the practical reason — more than the gradient — that attention replaced recurrence.

Vocabulary

unrolling	writing the recurrence out as a deep network with shared weights
BPTT	backpropagation applied to the unrolled network
truncated BPTT	unrolling only the last k steps, to bound cost and gradient
cell state	an LSTM's second, additively-updated memory
teacher forcing	feeding the true previous token during training
exposure bias	the mismatch that creates at prediction time
causal	no output depends on a later input

Scope

- the recurrent cell and what unrolling means; why BPTT is unstable and how gates address it; what each LSTM and GRU gate does; the three multi-step strategies; when a bidirectional layer is leakage — EXAMINABLE
- the framework's argument names, stateful mode, and the details of truncated BPTT — NOT EXAMINABLE — ENGINEERING
- attention, named today and derived in Lecture 18; probabilistic forecasting; hierarchical models — BEYOND THE SYLLABUS

If you can say why an additive update helps and a multiplicative one does not, you have the examinable half of this lecture.

What we learned

1. Weak stationarity is what a model assumes about a series\; ours does not have it
2. ∇^d removes a degree-d polynomial trend\; ∇_s removes a component of period s exactly
3. $\mathrm{Var}(y_t - y_{t-k}) = 2\sigma^2(1 - \rho_k)$: differencing helps if and only if $\rho_k > \frac{1}{2}$, and $\rho_7 = 0.831$ is the whole reason copying last week was hard to beat
4. A shuffled split of an autocorrelated series is biased optimistically, because the model is fitted on rows that surround what it is scored on
5. Measured here: 28% on the raw number, of which 9 to 10% survives every control — a fifth of the model's entire value over doing nothing
6. Repair with time-ordered splits and rolling-origin backtesting, and put a no-model baseline on identical rows beside every score
7. Then: multivariate inputs and a gated cell take the honest forecast from 64,815 to 34,639

In the book

Topic	Chapter
Stationarity, differencing, seasonality, moving averages	13
ARMA, ARIMA and SARIMA	13
Splitting a time series across time	13
Deep RNNs, multivariate inputs, several steps ahead, seq2seq	13
LSTM and GRU cells\; 1D convolution over sequences	13
Unstable gradients through depth	11
Cross-validation and its assumptions	2

What we did

1. Replaced a fixed window with a *state*, and unrolled the cell to see that a recurrent network is a deep network with shared weights
2. Saw that backpropagation through time is Lecture 11's problem with one aggravation: the same matrix multiplies the gradient at every step, so its powers vanish or explode
3. Met LSTM and GRU, and read each gate as an answer to that: a path along which the gradient is added to rather than multiplied
4. Forecast several steps ahead three ways — recursively, directly, and sequence-to-sequence — and saw what each assumes
5. Added multivariate inputs, and measured whether they paid
6. Saw a dilated convolution reach the same horizon without recurrence at all

One line to keep

> **THE LINE**
>
> **A gate is a path along which the gradient is added to rather than multiplied.** Everything else about LSTM and GRU is arrangement.

And its consequence, which is the honest summary of the lecture: gates make long-range dependence *trainable*, not *easy*. Position 400 is still four hundred steps away, and that is the problem Lecture 18 removes rather than mitigates.

Before the next lecture

- Run the notebook. Then lengthen the window and watch the plain RNN get *worse*, and the gradient probe say why
- Look at the forecast plot for a model that scores well. Check it is not the truth shifted by one step
- Lecture 17 changes the data from numbers to text and keeps the architecture. Almost everything today survives; what changes is what a “step” is

NOT PRESENTED — FOR AFTERWARDS

Exercises Lecture 16

five, in the style of the written paper

Exercises — Lecture 16

1 A recurrent network is trained on 56-day windows drawn from one series. Explain why shuffling the *windows* is legitimate while shuffling the *series* is not. [4]

2 The head of the recurrent model is a linear layer on the final hidden state. State why the hidden state alone will not do. [4]

3 Gates were introduced after a plain recurrent cell failed over long windows. State the mechanism, in terms of what happens to the same matrix over 56 steps. [4]

Solutions on Lecture 17’s deck.

Exercises — Lecture 16 (continued)

4 A colleague reports a large improvement on the ridership forecast after making the recurrent layer bidirectional. State what has happened, and the one question that settles it. [5]

5 Report the training MAE beside the test MAE. Name the two failures the pair distinguishes, and say why the test column alone cannot. [4]

Solutions on Lecture 17’s deck.

THE FIVE SET AT THE END OF LECTURE 15

Solutions Lecture 15

not part of this lecture

Solutions — Lecture 15

1. For a stationary series, $\mathrm{Var}(X_t - X_{t-h}) = 2\gamma(0)(1-\rho(h))$. State the condition on…

✓ $\rho(h) < 1/2$.

- The factor $2(1-\rho)$ exceeds 1 exactly then
- So the reflex 'it is not stationary, difference it' can make the problem harder

2. Explain why the seasonal-naive forecast is hard to beat on daily ridership, in terms of the autocorrelation…

✓ $\rho(7)$ is high, so the lag-7 difference is close to white noise — and copying last week is the model that assumes it is.

- The weekly cycle carries most of the structure
- What remains after removing it is close to unpredictable

Solutions — Lecture 15 (2)

3. State why MAPE is the wrong metric for transit ridership, using a specific day as the example.

✓ It divides by the truth, so an error on Christmas — a tenth of normal ridership — counts ten times an equal error on an ordinary weekday.

- The metric would spend capacity on the days nobody staffs for
- MAE in boardings is in the units the operations team already uses

4. A cross-validated forecast uses KFold(shuffle=True) and reports a good score. Name the two conditions the…

✓ No training row may come after a test row, and none may be adjacent to one.

- Shuffling puts a point's own future in the training set
- Neighbouring days are nearly the same number, so an adjacent row nearly gives away the answer

Solutions — Lecture 15 (3)

5. In March 2020 the series level falls by three quarters and the model stops working. State whether this is a…

✓ Neither — the protocol was correct and the world changed. It implies monitoring, with a written trigger.

- No split protects against a regime change
- A model never re-measured after deployment is an assumption wearing a number's clothes

LECTURE 17 ·\; GÉ\;RON CH. 14

Text

Softmax, cross-entropy and logits — derived, and why the loss consumes logits rather than probabilities
Subword tokenisation, and what an embedding space encodes
A recurrent classifier, then the same architecture with pretrained embeddings

Run the Lecture 16 notebook first